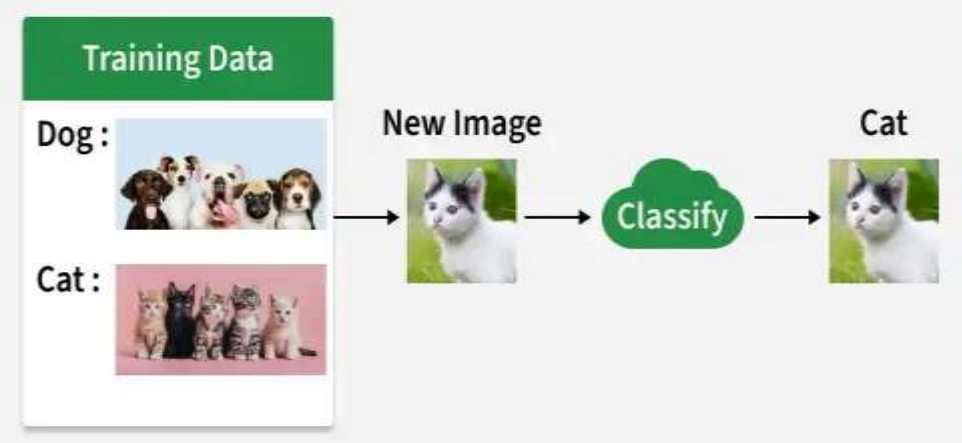
Day : 12 Classification

Classification is a supervised machine learning technique used to predict labels or categories from input data. It assigns each data point to a predefined class based on learned patterns.

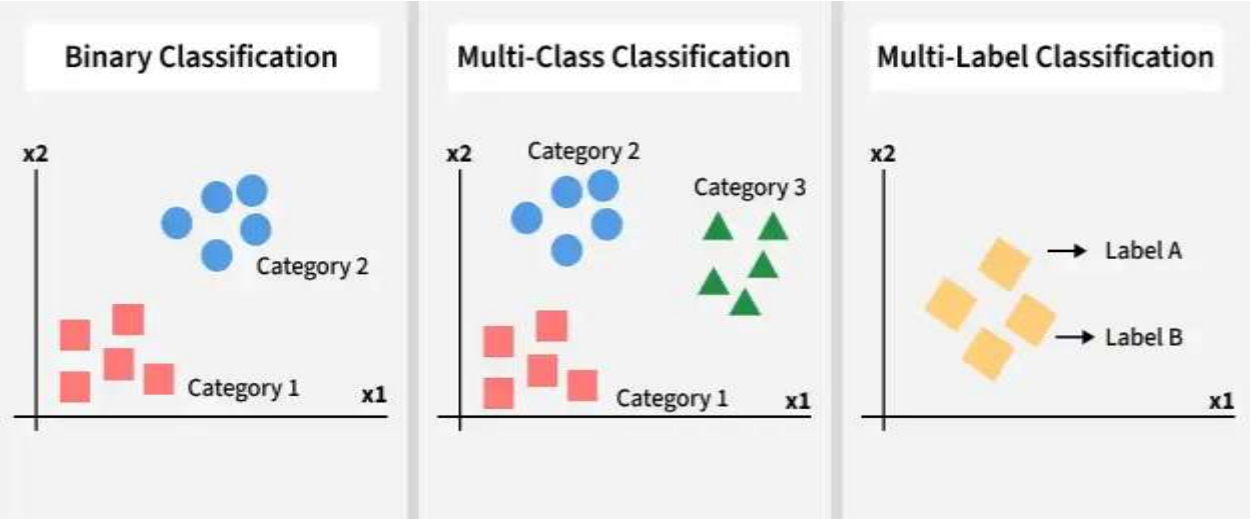
- Predict categories: Determines the class of new data points.
- Uses labeled data: Trained on datasets where the correct class is known.
- Common examples: Spam vs non spam emails, diseased vs. healthy patients.

For example: A classification model might be trained on dataset of images labeled as either dogs or cats and it can be used to predict the class of new and unseen images as dogs or cats based on their features such as colour, texture or shape.



Types of Classification

Classification in machine learning involves sorting data into categories based on their features or characteristics. The type of classification problem depends on how many classes exist and how the categories are structured.



Working

Classification works by training a model on labeled data so it can learn patterns and predict the correct class for new inputs. The main steps involved are:

1. **Data Collection:** Start with a dataset where each data point has a correct label.
2. **Feature Extraction**: Identify important features such as color, shape or texture that help distinguish classes.
3. **Model Training:** The algorithm learns patterns that connect features to the correct class.
4. **Model Evaluation:** The trained model is tested on unseen data to measure its accuracy.
5. **Prediction:** The model predicts the class of new data based on learned patterns.
6. **Model Improvement:** If performance is not satisfactory, the model or its parameters are adjusted and retrained.

Decision Tree Classifier:

A decision tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure which consists of a root node, branches, internal nodes and leaf nodes. It works like a flowchart that helps in making step-by-step decisions, where:

- Internal nodes represent attribute tests
- Branches represent attribute values
- Leaf nodes represent final decisions or predictions.

Decision trees are widely used due to their interpretability, flexibility and low preprocessing needs.

Implementation:

Step 1 — Import Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
```

Step 2 — Create the model

```
dt_classifier = DecisionTreeClassifier(
    random_state=42
)
```

Step 3 — Train it

Use the **same X_train, X_test, y_train, y_test** that you used for Logistic Regression:

```
dt_classifier.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

Step 4 — Make predictions

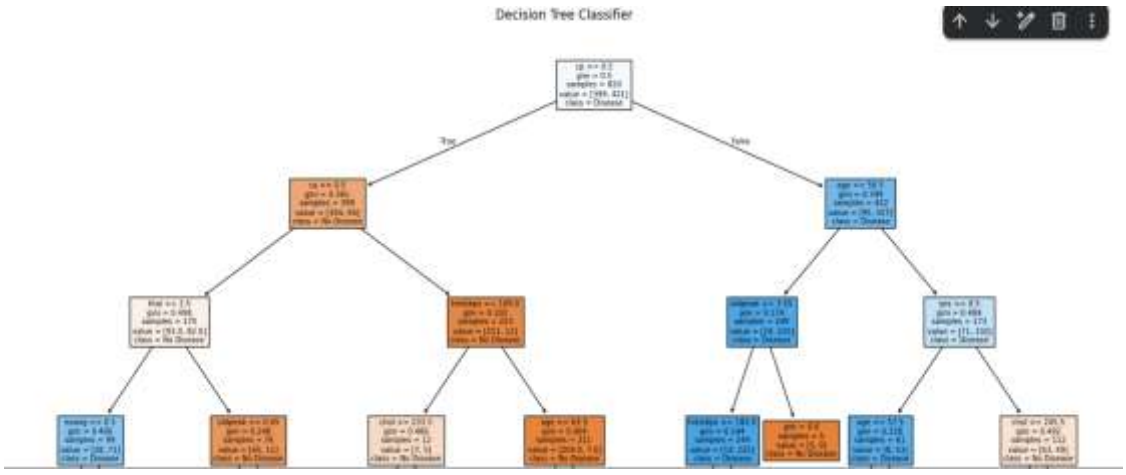
dt_pred = dt_classifier.predict(X_test)

Step 5 — Calculate accuracy

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, dt_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9853658536585366
```

```
from sklearn.tree import plot_tree
plt.figure(figsize=(20, 10))
plot_tree(
    dt_classifier,
    feature_names=X.columns,
    class_names=["No Disease", "Disease"],
    filled=True,
    max_depth=3
)
plt.title("Decision Tree Classifier")
plt.show()
```



Random Forest Classifier:

Random Forest is an ensemble machine learning algorithm that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting. In Scikit-learn, the Random Forest Classifier is widely used for classification tasks because it handles large datasets and handles nonlinear relationships well.

Working of Random Forest Classifier

1. Bootstrap Sampling: Random rows are picked (with replacement) to train each tree.
2. Random Feature Selection: Each tree uses a random set of features (not all features).
3. Build Decision Trees: Trees split the data using the best feature from their random set. Splitting continues until a stopping rule is met (like max depth).
4. Make Predictions: Each tree gives its own prediction.
5. Majority Voting: The final prediction is the one most tree agree on.

Implementing Random Forest Classification in Python

Before implementing random forest classifier in Python let's first understand it's parameters.

- n_estimators: Number of trees in the forest.
- max_depth: Maximum depth of each tree.
- max_features: Number of features considered for splitting at each node.
- criterion: Function used to measure split quality ('gini' or 'entropy').
- min_samples_split: Minimum samples required to split a node.
- min_samples_leaf: Minimum samples required to be at a leaf node.
- bootstrap: Whether to use bootstrap sampling when building trees (True or False).

Implementation:

Step 1 — Import

```
from sklearn.ensemble import RandomForestClassifier
```

Step 2 — Create the Random Forest model

```
rf_classifier = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

Step 3 — Train the model

```
rf_classifier.fit(X_train, y_train)
```

```
RandomForestClassifier(random_state=42)
```

Step 4 — Make predictions

rf_pred = rf_classifier.predict(X_test)

```
from sklearn.metrics import accuracy_score, confusion_matrix
print("Accuracy:", accuracy_score(y_test, rf_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, rf_pred))
```

```
Accuracy: 0.7540983606557377

Confusion Matrix:
[[20  8]
 [ 7 26]]
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, rf_pred))
```

```
              precision    recall  f1-score   support

           0       0.74      0.71      0.73        28
           1       0.76      0.79      0.78        33

    accuracy                           0.75        61
   macro avg       0.75      0.75      0.75        61
weighted avg       0.75      0.75      0.75        61
```

K Nearest Neighbour:

K-Nearest Neighbor (KNN) is a simple and widely used machine learning technique for classification and regression tasks. It works by identifying the *K* closest data points to a given input and making predictions based on the majority class or average value of those neighbors.

- Classifies data based on similarity with nearby data points
- Uses distance metrics like Euclidean distance to find nearest neighbors
- Since KNN makes no assumptions about the underlying data distribution, it makes it a non-parametric and instance-based learning method.

KNN is also called as a lazy learner algorithm because it does not learn from the training set immediately instead it stores the entire dataset and performs computations only at the time of classification.

How to choose the value of k for KNN Algorithm?

- The value of k in KNN decides how many neighbors the algorithm looks at when making a prediction.
- Choosing the right k is important for good results.
- If the data has lots of noise or outliers, using a larger k can make the predictions more stable.
- But if k is too large the model may become too simple and miss important patterns and this is called underfitting.
- So k should be picked carefully based on the data.

Implementation:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Load dataset

df = pd.read_csv('/content/heart.csv')

df_clean = df.drop_duplicates()

# 2. Separate features and target

X = df.drop(columns=['target'])

y = df['target']

# 3. Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y

)

# 4. Feature Scaling (CRITICAL for KNN)

# KNN calculates Euclidean distances, so unscaled features like cholesterol (~200)

# would dominate binary features like sex (0/1).

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# 5. Train KNN Model (default K=5)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
# 6. Evaluate initial model
y_pred = knn.predict(X_test_scaled)
print(f"Accuracy with K=5: {accuracy_score(y_test, y_pred):.4f}\n")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy with K=5: 0.8634

Confusion Matrix:
[[87 13]
 [15 90]]

Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.87      0.86       100
           1       0.87      0.86      0.87       105

    accuracy                           0.86       205
   macro avg       0.86      0.86      0.86       205
weighted avg       0.86      0.86      0.86       205
```